

Code Sample

Long Version

Anonymous

This document contains excerpts from the replication code of a paper titled “Inequality in a Potential Equalizer of Opportunity: Variation in the Accessibility and Net Benefits of English-Medium Instruction in India.” Excerpts are followed by their corresponding outputs when appropriate. This project may have been updated since this file was generated; the most up-to-date versions of this document, the full paper, and its replication package are available online for those interested. This PDF can be generated by running `make samples` or `scripts/run_full_build.sh` from the repo root.

Survey-Weighted Enrollment Model

The enrollment probit uses National Sample Survey (NSS) household weights, primary sampling units, and strata with a fixed set of child, household, and district covariates. The final analysis requires every variable used by the model and survey design, so incomplete inputs stop estimation instead of changing the specification.

File: `R/selection/estimate_selection_probit.R`

```
selection_probit_variables <- function(selection_data, require_all = FALSE) {  
  variables <- c(  
    "AGE", "SEX", "HH_SIZE", "RELIGION", "SOCIAL_GROUP", "SECTOR",  
    "DIST_FROM_NEAREST_PRIMARY_CLASS", "father_educ",  
    "dmean_num_IS_EDU_FREE", "dmean_num_TUTION_FEE_WAIVED",  
    "dmean_num_RECD_SCHOLARSHIP_STIPEND", "dmean_num_RECD_TXT_BOOKS",  
    "dmean_num_RECD_STATIONERY", "dmean_num_MID_DAY_MEAL_ETC_RECD",  
    "dmean_num_ENROLLMENT_COST"  
  )  
  
  missing <- setdiff(variables, names(selection_data))  
  if (isTRUE(require_all) && length(missing)) {
```

```

    stop(
      "Selection probit is missing required final analysis covariates: ",
      paste(missing, collapse = ", "),
      call. = FALSE
    )
  }
  variables[variables %in% names(selection_data)]
}

selection_survey_design_variables <- function() {
  c("FSU_SL_NO", "weight", "STATE", "SECTOR", "STRATUM", "SUB_STRATUM_NO")
}

## Retain the variables used by the selection model and survey design
##
## Earlier builds stored unrelated Block 5 schooling fields with the survey
## design. Editing one of those fields therefore recomputed the marginal effects
## even when the probit inputs were unchanged. Retain only variables used by the
## model and survey design.
project_selection_model_data <- function(selection_data) {
  selection_data <- safe_df(selection_data)
  design <- c("enrolled", selection_survey_design_variables())
  keep <- unique(c(design, selection_probit_variables(selection_data)))
  selection_data[intersect(keep, names(selection_data))]
}

## Estimate selection probit
##
estimate_selection_probit <- function(selection_data, cfg) {
  if (!"enrolled" %in% names(selection_data) || all(is.na(selection_data$enrolled))) {
    return(list(status = "out_of_active_pipeline", reason = "No enrolled variable."))
  }

  final_mode <- is_final_mode(cfg)
  covars <- selection_probit_variables(selection_data, require_all = final_mode)
  if (!length(covars)) {
    return(list(status = "out_of_active_pipeline", reason = "No probit covariates."))
  }
  f_probit <- stats::reformulate(covars, response = "enrolled")

  if (final_mode) {

```

```

    if (!requireNamespace("survey", quietly = TRUE)) {
      stop("The final selection probit requires the survey package.", call. = FALSE)
    }

    design <- build_survey_design_selection(selection_data, require_all = TRUE)
    out <- with_survey_lonely_psu(
      fit_selection_probit(design, f_probit)
    )
    return(stabilize_selection_model_formula(out, f_probit))
  }

  out <- stats::glm(
    f_probit,
    data = selection_data,
    family = stats::binomial(link = "probit")
  )
  stabilize_selection_model_formula(out, f_probit)
}

#' Store the fitted selection formula inside the saved model call
#'
#' Programmatic fitting originally left the local symbol `f_probit` in the saved
#' call. After {targets} serialized the model, packages reconstructing model data
#' could no longer resolve that local name. Store the formula itself in the call
#' and keep the formula attribute used by downstream validation.
stabilize_selection_model_formula <- function(model, formula) {
  if (!inherits(formula, "formula")) {
    stop("Selection model formula must inherit from formula.", call. = FALSE)
  }
  if (!is.null(model$call)) model$call$formula <- formula
  attr(model, "selection_probit_formula") <- formula
  model
}

#' Build survey design selection
#'
build_survey_design_selection <- function(selection_df, require_all = FALSE) {
  required <- selection_survey_design_variables()
  missing <- setdiff(required, names(selection_df))
  if (isTRUE(require_all) && length(missing)) {
    stop(
      "Selection survey design is missing required final analysis fields: ",

```

```

    paste(missing, collapse = ", "),
    call. = FALSE
  )
}

psu <- first_col(selection_df, c("FSU_SL_NO", "fsu", "PSU", "psu"))
weight <- first_col(selection_df, c("weight", "WEIGHT", "Multiplier", "multiplier"))
strata_cols <- intersect(c("STATE", "SECTOR", "STRATUM", "SUB_STRATUM_NO"), names(selection_df))
if (is.null(psu) || is.null(weight) || !length(strata_cols)) {
  if (isTRUE(require_all)) {
    stop("The final selection survey design could not be constructed.", call. = FALSE)
  }
  return(NULL)
}
selection_df$.survey_strata <- interaction(selection_df[strata_cols], drop = TRUE)
survey::svydesign(
  ids = stats::as.formula(paste0("~", psu)),
  strata = ~.survey_strata,
  weights = stats::as.formula(paste0("~", weight)),
  data = selection_df,
  nest = TRUE
)
}

#' Fit selection probit
#'
fit_selection_probit <- function(selection_design, f_probit) {
  survey::svyglm(
    f_probit,
    design = selection_design,
    family = stats::quasibinomial(link = "probit")
  )
}

```

Average Marginal Effects on the Estimation Sample

Average marginal effects are computed with `marginaleffects::avg_slopes()` over the observations used to fit the probit, weighted by their NSS sampling weights. The same calculation produces the enrollment table below.

File: R/selection/compute_average_marginal_effects.R

```

#' extract the fitted sample and survey weights used to average marginal effects
#'
ame_model_data_and_weights <- function(model) {
  model_data <- as.data.frame(stats::model.frame(model))

  model_weights <- if (inherits(model, "svyglm") && !is.null(model$survey.design)) {
    stats::weights(model$survey.design, type = "sampling")
  } else {
    tryCatch(stats::weights(model, type = "prior"), error = function(e) NULL)
  }

  if (is.null(model_weights)) model_weights <- rep(1, nrow(model_data))
  model_weights <- as.numeric(model_weights)
  if (length(model_weights) != nrow(model_data)) {
    stop("AME averaging weights do not align with the estimation sample.", call. = FALSE)
  }
  if (any(!is.finite(model_weights)) || any(model_weights < 0) || sum(model_weights) <= 0) {
    stop("AME averaging weights must be finite, nonnegative, and have positive total weight.", call. = FALSE)
  }

  list(data = model_data, wts = model_weights)
}

#' Run the standard marginaeffects implementation
#'
run_avg_slopes <- function(model, newdata = NULL, wts = TRUE, numberiv = NULL) {
  args <- list(
    model = model,
    newdata = newdata,
    wts = wts,
    vcov = TRUE,
    type = "response"
  )
  if (!is.null(numberiv)) args$numberiv <- numberiv
  do.call(marginaeffects::avg_slopes, args)
}

#' Compute AMEs with marginaeffects
#'
compute_ames_marginaeffects <- function(model) {
  # Earlier calls relied on marginaeffects to recover weights from serialized

```

```

# svyglm objects. Some saved models no longer exposed those weights reliably,
# so pass the fitted model frame and sampling weights explicitly. This keeps the
# averaging population equal to the survey estimation sample. `marginaleffects`
# uses the model's design-based covariance matrix for uncertainty.

amed <- ame_model_data_and_weights(model)
run_avg_slopes(model, newdata = amed$data, wts = amed$wts)
}

```

Table 7: Average Marginal Effects for School Enrollment

Term	Enrolled (1 = yes)
Age (years)	-0.028*** (0.000)
Female (ref: Male)	0.044*** (0.003)
Household size	-0.004*** (0.001)
Religion: Muslim (ref: Hindu)	-0.126*** (0.007)
Religion: Christian	-0.043*** (0.013)
Religion: Sikh	0.040 (0.026)
Religion: Jain	-0.001 (0.012)
Religion: Buddhist	0.047* (0.024)
Religion: Zoroastrian	0.101*** (0.034)
Religion: Other	0.252*** (0.003)
Social group: Scheduled Tribe (ref: Other)	-0.102*** (0.008)
Social group: Scheduled Caste	-0.075*** (0.006)
Social group: Other Backward Class	-0.029*** (0.005)
Urban (ref: Rural)	-0.003 (0.005)
Distance 1–2km (ref: <1km)	-0.077*** (0.026)
Distance 2–3km	-0.001 (0.010)
Distance 3–5km	0.004 (0.066)
Distance > 5km	0.036 (0.035)
Male household education: Literate, no school (ref: Illiterate)	0.097*** (0.017)
Male household education: Literate, school < primary	0.127*** (0.006)
Male household education: Primary	0.142*** (0.005)

Table 7: Average Marginal Effects for School Enrollment (*continued*)

Term	Enrolled (1 = yes)
Male household education: Upper primary	0.195*** (0.006)
Male household education: Secondary	0.212*** (0.007)
Male household education: Higher secondary	0.247*** (0.008)
Male household education: Postsecondary+	0.245*** (0.009)

* $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$. Entries are average changes in predicted enrollment probability. Continuous covariates are average slopes; categorical covariates are average discrete comparisons against the stated reference category, evaluated over the observed covariate distribution. Standard errors use the NSS sampling design, including household weights, primary sampling units, and strata formed from state, stratum, and substratum identifiers. The model also controls for district averages of schooling costs and benefits among enrolled children; the table displays marginal effects for child and household covariates. Source: NSS 64th Round, 2007–08, Participation and Expenditure in Education.

Reconstructing District Boundary Transfers Between 1991 and 2001

A headerless historical table records how the population of each 1991 district was divided among 2001 districts. Wrapped district names are joined only when the row structure identifies a continuation; every transfer percentage is checked, and the shares leaving each 1991 district must sum to 100 within rounding tolerance.

File: R/io/read_inputs.R

```
#' Read the headerless table of district boundary changes from 1961 to 2001
#
#' The historical table has five data columns and no header row. Reading it with
#' the ordinary CSV default would consume the first Anantapur observation as names.
repair_district_carveout_wrapped_rows <- function(x) {
  out <- safe_df(x)
  if (!nrow(out)) return(out)
  blank <- function(value) is.na(value) || !nzchar(trimws(as.character(value)))
  drop <- rep(FALSE, nrow(out))
  for (i in seq_len(nrow(out))) {
    if (i == 1L || !blank(out$pop_1991[[i]])) next
    previous <- i - 1L
    source_continuation <- !blank(out$district_1991[[i]])
    target_continuation <- !blank(out$district_2001[[i]])
    has_transfer <- !blank(out$pct_01in91[[i]]) || !blank(out$pct_91in01[[i]])
```

```

if (source_continuation) {
  previous_name <- plain_chr(out$district_1991[[previous]])
  if (!length(previous_name) || !grepl("-", trimws(previous_name[[1L]]))) {
    stop("Unexpected wrapped row in the district boundary table.", call. = FALSE)
  }
  continuation <- plain_chr(out$district_1991[[i]])[[1L]]
  combined <- join_district_carveout_wrapped_label(previous_name[[1L]], continuation)
  out$district_1991[[previous]] <- combined
  if (has_transfer) {
    out$district_1991[[i]] <- combined
    out$pop_1991[[i]] <- out$pop_1991[[previous]]
  }
}

if (!has_transfer && (source_continuation || target_continuation)) {
  if (target_continuation) {
    if (blank(out$district_2001[[previous]])) {
      stop("Unexpected continuation of a 2001 district name in the district boundary table.", call. =
        FALSE)
    }
    previous_target <- trimws(as.character(out$district_2001[[previous]]))
    continuation <- trimws(as.character(out$district_2001[[i]]))
    out$district_2001[[previous]] <- join_district_carveout_wrapped_label(
      previous_target, continuation
    )
  }
  drop[[i]] <- TRUE
}
out[!drop, , drop = FALSE]
}

district_carveout_rounding_tolerance_pp <- function() 0.05

validate_district_carveout_shares <- function(
  x,
  rounding_tolerance_pp = district_carveout_rounding_tolerance_pp()) {
  out <- safe_df(x)
  if (!nrow(out)) return(invisible(TRUE))
  share_cols <- c("pct_01in91", "pct_91in01")
  for (field in share_cols) {

```

```

values <- num(out[[field]])
bad <- is.finite(values) & (values < 0 | values > 100)
if (any(bad)) {
  stop("District transfer shares must lie in [0, 100].", call. = FALSE)
}
}
source_share <- num(out$pct_01in91)
if (any(!is.finite(source_share))) {
  stop("Shares of each 1991 district assigned to 2001 districts must be complete.", call. = FALSE)
}
source_key <- paste(out$district_1991, out$pop_1991, sep = "__")
source_sum <- vapply(split(source_share, source_key), sum, numeric(1))
if (any(abs(source_sum - 100) > rounding_tolerance_pp)) {
  stop("Shares of each 1991 district assigned to 2001 districts must sum to 100 within rounding
  ↪ tolerance.", call. = FALSE)
}
invisible(TRUE)
}

read_district_carveouts <- function(path) {
  out <- utils::read.csv(
    path,
    header = FALSE,
    col.names = c("district_1991", "pop_1991", "district_2001", "pct_01in91", "pct_91in01"),
    stringsAsFactors = FALSE,
    na.strings = c("", "NA"),
    check.names = FALSE
  )
  out <- repair_district_carveout_wrapped_rows(out)
  out$district_1991 <- fill_down_missing(out$district_1991)
  out$pop_1991 <- num(gsub(",", "", fill_down_missing(out$pop_1991), fixed = TRUE))
  out$pct_01in91 <- num(out$pct_01in91)
  out$pct_91in01 <- num(out$pct_91in01)
  validate_district_carveout_shares(out)
  out
}

```

Reconstructing District Data from the 1991 Language Atlas

Annexure IV of the 1991 Language Atlas repeats the same district rows across eight language pages. Exact repeated labels can supply a district serial number when one page omits it; extracted district populations are

then compared with the official 1991 Primary Census Abstract (PCA), and disagreements are flagged for review.

File: scripts/build_language_atlas_1991.py

```
def cross_page_serial_evidence(cells: list[dict[str, object]]) -> dict[tuple[int, str], tuple[int, ...]]:
    """Collect district serials printed beside the same Atlas row label.

    Annexure IV repeats each district row on eight language pages. An exact label
    repeated with one serial can therefore supply a serial omitted on another page.
    Conflicting printed serials remain unresolved.
    """
    observed: dict[tuple[int, str], set[int]] = {}
    unique_rows = {
        (int(cell["block"]), int(cell["page"]), int(cell["row_sequence"]),
         str(cell["row_label_raw"]).strip(), str(cell["serial_candidate"]).strip())
        for cell in cells
    }
    for block, _page, _sequence, label, serial_raw in unique_rows:
        if not label or label == "INDIA":
            continue
        serial = int(serial_raw) if serial_raw else parse_leading_serial(label)
        if serial is not None:
            observed.setdefault((block, label), set()).add(serial)
    return {key: tuple(sorted(values)) for key, values in observed.items()}

def reconcile_district_serial(
    cell: dict[str, object],
    evidence: dict[tuple[int, str], tuple[int, ...]],
) -> tuple[int | None, str, str]:
    """Resolve one Atlas district serial using direct and exact-label evidence."""
    label = str(cell["row_label_raw"]).strip()
    serial_column = int(cell["serial_candidate"]) if str(cell["serial_candidate"]).strip() else None
    serial_label = parse_leading_serial(label)
    repeated = evidence.get((int(cell["block"]), label), ())
    repeated_text = ";".join(map(str, repeated))

    if serial_column is not None and serial_label is not None and serial_column != serial_label:
        return None, "direct_conflict", repeated_text
    direct = serial_column if serial_column is not None else serial_label
    direct_source = "serial_column" if serial_column is not None else "label_prefix"
```

```

if direct is not None:
    # Use a serial printed on this row when it is available. Exact labels on
    # repeated language pages supply a serial only when this field is missing;
    # continuation pages occasionally contain isolated OCR errors.
    return direct, direct_source, repeated_text
if len(repeated) == 1:
    return repeated[0], "cross_page_exact_label", repeated_text
if len(repeated) > 1:
    return None, "cross_page_conflict", repeated_text
return None, "missing", repeated_text

def build_district_population_validation(
    cells: list[dict[str, object]],
    state_crosswalk: dict[str, dict[str, str]],
    pca91: dict[tuple[str, str], int],
) -> list[dict[str, object]]:
    rows = merge_population_continuations(cells, state_crosswalk)
    serial_evidence = cross_page_serial_evidence(cells)
    seen_state_codes: set[str] = set()
    current_state: dict[str, str] | None = None
    candidates: list[dict[str, object]] = []
    for cell in rows:
        label = str(cell["row_label_raw"]).strip()
        if label == "INDIA":
            continue
        mapped_state = state_crosswalk.get(label)
        if mapped_state is not None and mapped_state["state_code_1991"] not in seen_state_codes:
            current_state = mapped_state
            seen_state_codes.add(mapped_state["state_code_1991"])
            continue

        serial, serial_source, cross_page_serials = reconcile_district_serial(cell, serial_evidence)

        state_code = current_state["state_code_1991"] if current_state else ""
        state_name = current_state["state_name_1991"] if current_state else ""
        district_code = f"{serial:02d}" if serial is not None else ""
        pca_population = pca91.get((state_code, district_code)) if state_code and district_code else None
        atlas_population = (
            int(cell["speaker_count_candidate"])

```

```

        if str(cell["speaker_count_candidate"]).strip()
        else None
    )
    if atlas_population is not None and pca_population is not None and pca_population > 0:
        abs_diff = abs(atlas_population - pca_population)
        rel_diff = abs_diff / pca_population
    else:
        abs_diff = None
        rel_diff = None
    candidates.append({
        "block": cell["block"],
        "page": cell["page"],
        "row_sequence": cell["row_sequence"],
        "row_sequence_end": cell["row_sequence_end"],
        "row_merge_status": cell["row_merge_status"],
        "row_label_raw": label,
        "state_code_1991": state_code,
        "state_name_1991": state_name,
        "district_code_1991": district_code,
        "serial_source": serial_source,
        "cross_page_serials": cross_page_serials,
        "atlas_population_raw": cell["raw_value"],
        "atlas_population_candidate": "" if atlas_population is None else atlas_population,
        "pca91_population": "" if pca_population is None else pca_population,
        "population_abs_diff": "" if abs_diff is None else abs_diff,
        "population_relative_diff": "" if rel_diff is None else round(rel_diff, 8),
        "identity_status": "",
        "population_status": "",
        "promotion_status": "review_required",
    })

expected_state_codes = {row["state_code_1991"] for row in state_crosswalk.values()}
if seen_state_codes != expected_state_codes:
    missing = sorted(expected_state_codes - seen_state_codes)
    extra = sorted(seen_state_codes - expected_state_codes)
    raise ValueError(
        "Language Atlas state-heading coverage mismatch; "
        f"missing={missing}; extra={extra}"
    )

pca_state_codes = {state_code for state_code, _ in pca91}
if pca_state_codes != expected_state_codes:

```

```

    raise ValueError("Language Atlas state crosswalk and SHRUG PCA91 state codes disagree")

key_counts: dict[tuple[str, str], int] = {}
for row in candidates:
    key = (str(row["state_code_1991"]), str(row["district_code_1991"]))
    if all(key):
        key_counts[key] = key_counts.get(key, 0) + 1

for row in candidates:
    key = (str(row["state_code_1991"]), str(row["district_code_1991"]))
    if not row["state_code_1991"]:
        identity_status = "missing_state_context"
    elif row["serial_source"] in {"direct_conflict", "cross_page_conflict"}:
        identity_status = "serial_conflict"
    elif not row["district_code_1991"]:
        identity_status = "missing_district_serial"
    elif key_counts.get(key, 0) > 1:
        identity_status = "duplicate_district_code"
    elif key not in pca91:
        identity_status = "missing_pca_district"
    else:
        identity_status = "candidate_official_code"
    row["identity_status"] = identity_status

    if row["atlas_population_candidate"] == "":
        population_status = "atlas_population_unparsed"
    elif row["pca91_population"] == "":
        population_status = "pca_population_missing"
    elif float(row["population_relative_diff"]) <= POPULATION_RELATIVE_TOLERANCE:
        population_status = "within_1pct"
    else:
        population_status = "over_1pct"
    row["population_status"] = population_status
    if identity_status == "candidate_official_code" and population_status == "within_1pct":
        row["promotion_status"] = "population_validated_candidate"
return candidates

```

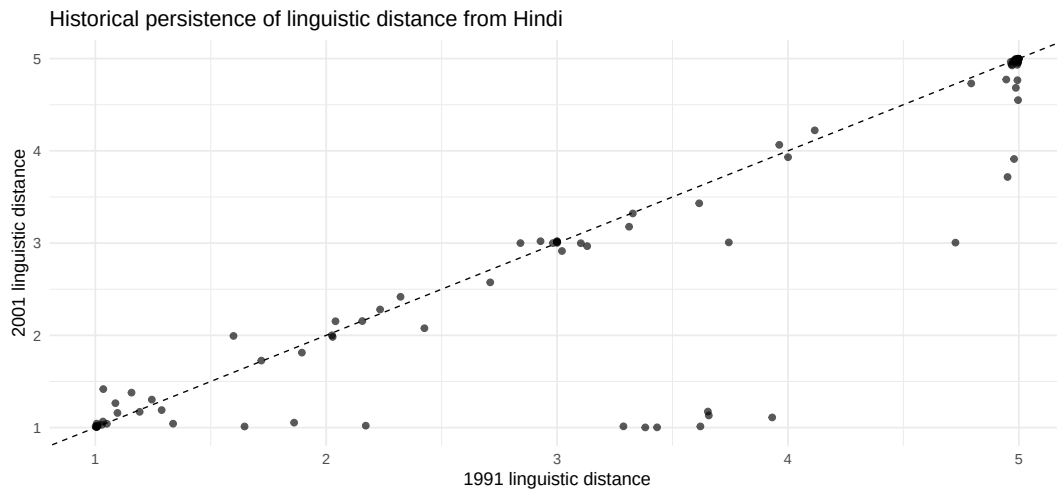


Figure 2: Persistence of speaker-weighted linguistic distance from Hindi from 1991 to 2001. Each point is a 2001 Census district with a reconstructed measure in both years; the dashed line is the 45-degree line. Sources: Census of India 1991 Language Atlas (Office of the Registrar General and Census Commissioner, India 2004) and Census of India 2001 language tables (Office of the Registrar General and Census Commissioner of India 2001).

Rook and Queen Contiguity from District Polygons

Rook and queen neighbors are constructed directly from district polygons with `spdep` while retaining the fitted sample's row order. The routine records islands and connected components, allowing genuine offshore districts to be distinguished from unexpected disconnected components.

File: `R/diagnostics/diagnose_spatial_weights.R`

```
build_spatial_weights_for_rows <- function(district_panel, rows, queen = FALSE, snap = NULL) {
  if (!inherits(district_panel, "sf")) {
    return(list(status = "out_of_active_pipeline", reason = "Requires sf geometry."))
  }
  need_pkg("spdep", "spatial weights")
  need_pkg("sf", "spatial weights")

  rows <- as.integer(rows)
  rows <- rows[is.finite(rows) & rows >= 1L & rows <= nrow(district_panel)]
  if (!length(rows)) {
    return(list(status = "out_of_active_pipeline", reason = "No rows with usable geometry."))
  }
  geom <- sf::st_geometry(district_panel)
```

```

rows <- rows[!sf::st_is_empty(geom[rows])]
if (!length(rows)) {
  return(list(status = "out_of_active_pipeline", reason = "Selected rows have empty geometry."))
}

spatial_warnings <- character()
capture_expected_spatial_warning <- function(expr) {
  withCallingHandlers(
    expr,
    warning = function(w) {
      msg <- conditionMessage(w)
      expected <- grepl("no neighbours|sub-graphs", msg, ignore.case = TRUE)
      if (expected) {
        spatial_warnings <-< unique(c(spatial_warnings, msg))
        invokeRestart("muffleWarning")
      }
    }
  )
}

panel <- district_panel[rows, , drop = FALSE]
nb <- capture_expected_spatial_warning(poly2nb_with_optional_snap(panel, queen = queen, snap = snap))
W <- capture_expected_spatial_warning(spdep::nb2mat(nb, style = "B", zero.policy = TRUE))
listw <- capture_expected_spatial_warning(spdep::nb2listw(nb, style = "W", zero.policy = TRUE))

neighbor_counts <- spdep::card(nb)
n_components <- spdep::n.comp.nb(nb)$nc %||% NA_integer_
snap_used <- if (is.null(snap)) NA_real_ else as.numeric(snap)
panel_data <- as.data.frame(sf::st_drop_geometry(panel), stringsAsFactors = FALSE)
identifier_cols <- intersect(
  c(
    "district_panel_id", "state_code_2001", "district_code_2001",
    "state_20", "district_20", "state_std", "district_std"
  ),
  names(panel_data)
)
neighbor_ledger <- data.frame(
  row_index = rows,
  n_neighbors = neighbor_counts,
  panel_data[identifier_cols],
  check.names = FALSE,

```

```

    stringsAsFactors = FALSE
  )

  offshore_islands_expected <- spatial_expected_offshore_islands(
    neighbor_ledger, n_components
  )

  disconnected <- sum(neighbor_counts == 0L) > 0L ||
    (!is.na(n_components) && n_components > 1L)

  out <- list(
    status = "constructed",
    contiguity = if (isTRUE(queen)) "queen" else "rook",
    style = "W",
    matrix_style = "B",
    zero_policy = TRUE,
    row_index = rows,
    nb = nb,
    W = W,
    listw = listw,
    neighbor_counts = neighbor_counts,
    neighbor_ledger = neighbor_ledger,
    n = length(rows),
    n_islands = sum(neighbor_counts == 0L),
    mean_neighbors = mean(neighbor_counts),
    n_subgraphs = n_components,
    snap = snap_used,
    offshore_islands_expected = offshore_islands_expected,
    snap_investigation_needed = disconnected && !offshore_islands_expected,
    warnings = spatial_warnings
  )

  class(out) <- c("emi_spatial_weights", class(out))
  out
}

```

Moran's I Permutation Test for Regression Residuals

Moran's I is computed on the observations used to fit each model. The code reports both the analytical test and a permutation test with 9,999 randomizations from `spdep::moran.mc()` using a fixed random seed; the paper uses the permutation p-value for regression residuals.

File: `R/diagnostics/diagnose_spatial_autocorrelation.R`

```

spatial_moran_test_from_model_residuals <- function(model, district_panel, weights, legacy_name, estimand,
  ↪ variable, source) {
  if (!inherits(model, "ivreg")) {
    return(spatial_autocorrelation_status_row("out_of_active_pipeline", paste0("No active IV model for ",
  ↪ estimand, ".")))
  }
  rows <- spatial_model_rows(model, district_panel)
  if (!identical(as.integer(rows), as.integer(weights$row_index))) {
    weights <- build_spatial_weights_for_rows(
      district_panel, rows, queen = identical(weights$contiguity, "queen")
    )
  }
  x <- tryCatch(stats::residuals(model), error = function(e) NA_real_)
  compute_moran_tests(
    x, weights, legacy_name, estimand, variable, source, run_permutation = TRUE
  )
}

```

```

spatial_moran_test_from_first_stage_residuals <- function(model, district_panel, weights, legacy_name,
  ↪ estimand, variable, source) {
  if (!inherits(model, "ivreg")) {
    return(spatial_autocorrelation_status_row(
      "out_of_active_pipeline",
      paste0("No fitted IV first stage is available for ", estimand, ".")
    ))
  }
  rows <- spatial_model_rows(model, district_panel)
  residuals <- tryCatch(
    stats::residuals(model, type = "stage1"),
    error = function(e) NULL
  )
  if (is.null(residuals) || !length(rows)) {
    return(spatial_autocorrelation_status_row(
      "out_of_active_pipeline",
      paste0("Could not recover fitted first-stage residuals for ", estimand, ".")
    ))
  }
  residuals <- as.matrix(residuals)
  if (ncol(residuals) != 1L || nrow(residuals) != length(rows)) {
    return(spatial_autocorrelation_status_row(
      "out_of_active_pipeline",

```

```

    paste0("The spatial first-stage residual test requires one endogenous regressor for ", estimand, ".")
  ))
}
if (!identical(as.integer(rows), as.integer(weights$row_index))) {
  weights <- build_spatial_weights_for_rows(
    district_panel, rows, queen = identical(weights$contiguity, "queen")
  )
}
compute_moran_tests(
  residuals[, 1L], weights, legacy_name, estimand, variable, source,
  run_permutation = TRUE
)
}

#' Compute Moran's I tests
#'
#' @return A one-row data frame containing Moran's I and the analytical
#' randomisation test result from `spdep::moran.test()`. For regression
#' residuals, the returned row also contains the permutation p-value computed with a fixed random seed
#' from `spdep::moran.mc()`. Descriptive variable tests use the analytical test.
compute_moran_tests <- function(
  x, spatial_weights, legacy_name = NA_character_, estimand = NA_character_,
  variable = NA_character_, source = NA_character_, run_permutation = FALSE,
  permutation_nsim = 9999L, permutation_seed = 1L) {
  if (!inherits(spatial_weights, "emi_spatial_weights") || !identical(spatial_weights$status, "constructed"))
    ↪ {
      return(spatial_autocorrelation_status_row("out_of_active_pipeline", "Spatial weights were not
        ↪ constructed."))
    }
  x <- suppressWarnings(as.numeric(x))
  if (length(x) != length(spatial_weights$row_index)) {
    return(spatial_autocorrelation_status_row("out_of_active_pipeline", paste0("Length mismatch for ",
      ↪ variable, ": ", length(x), " values for ", length(spatial_weights$row_index), " spatial rows.")))
  }
  if (any(!is.finite(x))) {
    return(spatial_autocorrelation_status_row("out_of_active_pipeline", paste0("Non-finite values prevent
      ↪ Moran's I for ", variable, ".")))
  }
  run_permutation <- isTRUE(run_permutation)
  permutation_nsim <- as.integer(permutation_nsim)
  permutation_seed <- as.integer(permutation_seed)

```

```

if (run_permutation && (length(permutation_nsim) != 1L || !is.finite(permutation_nsim) || permutation_nsim
↪ < 1L)) {
  stop("Moran permutation count must be one positive integer.", call. = FALSE)
}
if (run_permutation && (length(permutation_seed) != 1L || !is.finite(permutation_seed))) {
  stop("Moran permutation seed must be one finite integer.", call. = FALSE)
}

test <- tryCatch(
  spdep::moran.test(x, spatial_weights$listw, zero.policy = TRUE),
  error = function(e) e
)
if (inherits(test, "error")) {
  return(spatial_autocorrelation_status_row("out_of_active_pipeline", conditionMessage(test)))
}
permutation_test <- NULL
if (run_permutation) {
  permutation_test <- tryCatch(
    withr::with_seed(
      permutation_seed,
      spdep::moran.mc(
        x, spatial_weights$listw, nsim = permutation_nsim, zero.policy = TRUE
      )
    ),
    error = function(e) e
  )
  if (inherits(permutation_test, "error")) {
    return(spatial_autocorrelation_status_row(
      "out_of_active_pipeline",
      paste0("Moran permutation test failed: ", conditionMessage(permutation_test))
    ))
  }
}

estimate <- suppressWarnings(as.numeric(test$estimate))
estimate_value <- function(i) if (length(estimate) >= i && is.finite(estimate[[i]])) estimate[[i]] else
↪ NA_real_
data.frame(
  legacy_name = legacy_name,
  estimand = estimand,
  variable = variable,

```

```

source = source,
test = "moran",
status = "estimated",
statistic = suppressWarnings(as.numeric(test$statistic[[1]])),
estimate = estimate_value(1L),
expected = estimate_value(2L),
variance = estimate_value(3L),
p.value = suppressWarnings(as.numeric(test$p.value)),
method = test$method %||% "Moran I test under randomisation",
alternative = test$alternative %||% NA_character_,
permutation_p.value = if (is.null(permutation_test)) NA_real_ else
  ↪ suppressWarnings(as.numeric(permutation_test$p.value)),
permutation_method = if (is.null(permutation_test)) NA_character_ else permutation_test$method %||%
  ↪ "Monte-Carlo simulation of Moran I",
permutation_nsim = if (is.null(permutation_test)) NA_integer_ else permutation_nsim,
permutation_seed = if (is.null(permutation_test)) NA_integer_ else permutation_seed,
n = length(x),
contiguity = spatial_weights$contiguity %||% NA_character_,
weights_style = spatial_weights$style %||% NA_character_,
matrix_style = spatial_weights$matrix_style %||% NA_character_,
zero_policy = spatial_weights$zero_policy %||% NA,
n_spatial_rows = spatial_weights$n %||% length(x),
n_islands = spatial_weights$n_islands %||% NA_integer_,
mean_neighbors = spatial_weights$mean_neighbors %||% NA_real_,
warnings = paste(spatial_weights$warnings %||% character(), collapse = "; "),
reason = NA_character_,
stringsAsFactors = FALSE
)
}

```

Mapping District Estimates on 2001 Census Boundaries

Each statistic is joined to the complete 2001 Census district boundary layer, leaving districts outside a particular estimation sample visible as missing data. Disputed territorial polygons are shown as no data, state outlines are reconstructed from the complete boundary layer, and the same functions draw categorical or numeric choropleths, including diverging scales.

File: R/output/save_figures.R

```

build_public_ggplot_map <- function(plot_data, spec, boundary_reference = NULL) {
  need_pkg("ggplot2", "classified choropleth maps")
  style <- public_map_style(spec$variable)
  fill <- public_map_fill(plot_data, spec$variable, style)
  plot_data <- fill$data
  overlay <- plot_data[map_overlay_rows(plot_data, fill$fill), , drop = FALSE]
  if (!nrow(overlay)) {
    stop("Map figure '", spec$name, "' has no non-missing overlay districts for variable '", spec$variable,
      ↪ "'.", call. = FALSE)
  }

  disputed_display <- public_map_disputed_display(plot_data, boundary_reference)
  display_data <- mask_public_map_disputed_areas(plot_data, disputed_display)
  state_outlines <- public_map_state_outlines(display_data)
  base <- ggplot2::ggplot() +
    ggplot2::geom_sf(
      data = display_data, ggplot2::aes(fill = .data[[fill$fill]]),
      color = "grey55", linewidth = map_district_boundary_linewidth(),
      linetype = "solid"
    )
  if (!is.null(disputed_display) && nrow(disputed_display)) {
    base <- base + ggplot2::geom_sf(
      data = disputed_display, fill = map_disputed_no_data_colour(),
      color = "grey45", linewidth = map_disputed_boundary_linewidth(),
      linetype = "solid"
    )
  }
  if (!is.null(state_outlines) && nrow(state_outlines)) {
    base <- base + ggplot2::geom_sf(
      data = state_outlines, color = "grey15",
      linewidth = map_major_boundary_linewidth(), linetype = "solid"
    )
  }
  base <- base +
    ggplot2::coord_sf(datum = NA) +
    ggplot2::labs(fill = fill$title) +
    ggplot2::theme_void(base_size = 10) +
    ggplot2::theme(
      legend.position = "right",
      legend.title = ggplot2::element_text(size = 12, face = "bold"),

```

```

    legend.text = ggplot2::element_text(size = 10),
    legend.key.height = grid::unit(16, "pt"),
    legend.spacing.y = grid::unit(4, "pt"),
    plot.margin = grid::unit(c(2, 2, 2, 2), "pt")
  )

  if (isTRUE(fill$continuous)) {
    return(base + ggplot2::scale_fill_gradientn(
      colours = fill$colors,
      limits = fill$limits,
      na.value = map_no_data_colour(),
      guide = public_map_colorbar_guide()
    ))
  }

  base +
    ggplot2::scale_fill_manual(
      values = fill$colors,
      breaks = names(fill$colors),
      limits = names(fill$colors),
      drop = FALSE,
      na.translate = TRUE,
      na.value = map_no_data_colour()
    ) +
    ggplot2::guides(
      fill = ggplot2::guide_legend(
        override.aes = map_legend_override(fill$colors)
      )
    )
}

complete_public_map_geometry <- function(district_panel, map_geometry) {
  if (!has_sf_geometry(map_geometry)) {
    stop("District maps require the complete 2001 Census district geometry.", call. = FALSE)
  }

  key <- "target_unit_2001"

  if (!key %in% names(map_geometry) && "unit_id" %in% names(map_geometry)) {
    map_geometry[[key]] <- map_geometry$unit_id
  }

  if (!key %in% names(map_geometry) || !key %in% names(district_panel)) {
    stop("Map geometry and analysis data must contain a 2001 Census district key.", call. = FALSE)
  }
}

```

```

}
if (anyDuplicated(map_geometry[[key]])) {
  stop("Complete 2001 Census map geometry must contain one row per district.", call. = FALSE)
}
attributes <- if (inherits(district_panel, "sf")) sf::st_drop_geometry(district_panel) else
↪ safe_df(district_panel)
attributes <- attributes[!duplicated(attributes[[key]]), , drop = FALSE]
out <- merge(map_geometry, attributes, by = key, all.x = TRUE, sort = FALSE)

# Derive state identity from the complete 2001 Census boundary identifier so
# state and national outlines include districts with missing analysis values.
# The district identifier stores the 2001 state code directly.
boundary_state <- public_map_state_code_2001(out[[key]])
if ("state_code_2001" %in% names(out)) {
  panel_state <- plain_chr(out$state_code_2001)
  mismatch <- !is.na(panel_state) & nzchar(panel_state) & panel_state != boundary_state
  if (any(mismatch)) {
    stop("Analysis state codes disagree with the state codes in the 2001 Census district identifiers.",
↪ call. = FALSE)
  }
}
out$state_code_2001 <- boundary_state
out
}

save_map_plot_formats <- function(map_plot, path_base, formats, width = 8, height = 6, dpi = 300) {
  save_plot_formats(map_plot, path_base, formats, width = width, height = height, dpi = dpi)
}

save_map_figure <- function(spec, path_base, district_panel, map_geometry, boundary_reference, formats) {
  if (!has_sf_geometry(district_panel)) {
    stop("Map figure '", spec$name, "' requires an sf district_panel with validated geometry.", call. =
↪ FALSE)
  }
  if (is.null(spec$variable) || !spec$variable %in% names(district_panel)) {
    stop("Map figure '", spec$name, "' is missing variable '", spec$variable, "'.", call. = FALSE)
  }

  plot_data <- complete_public_map_geometry(district_panel, map_geometry)
  plot_data <- prepare_public_map_data(plot_data, spec$variable)
  p <- build_public_ggplot_map(plot_data, spec, boundary_reference = boundary_reference)

```

```

save_map_plot_formats(p, path_base, formats, width = 7.2, height = 5.2, dpi = 300)
}

```

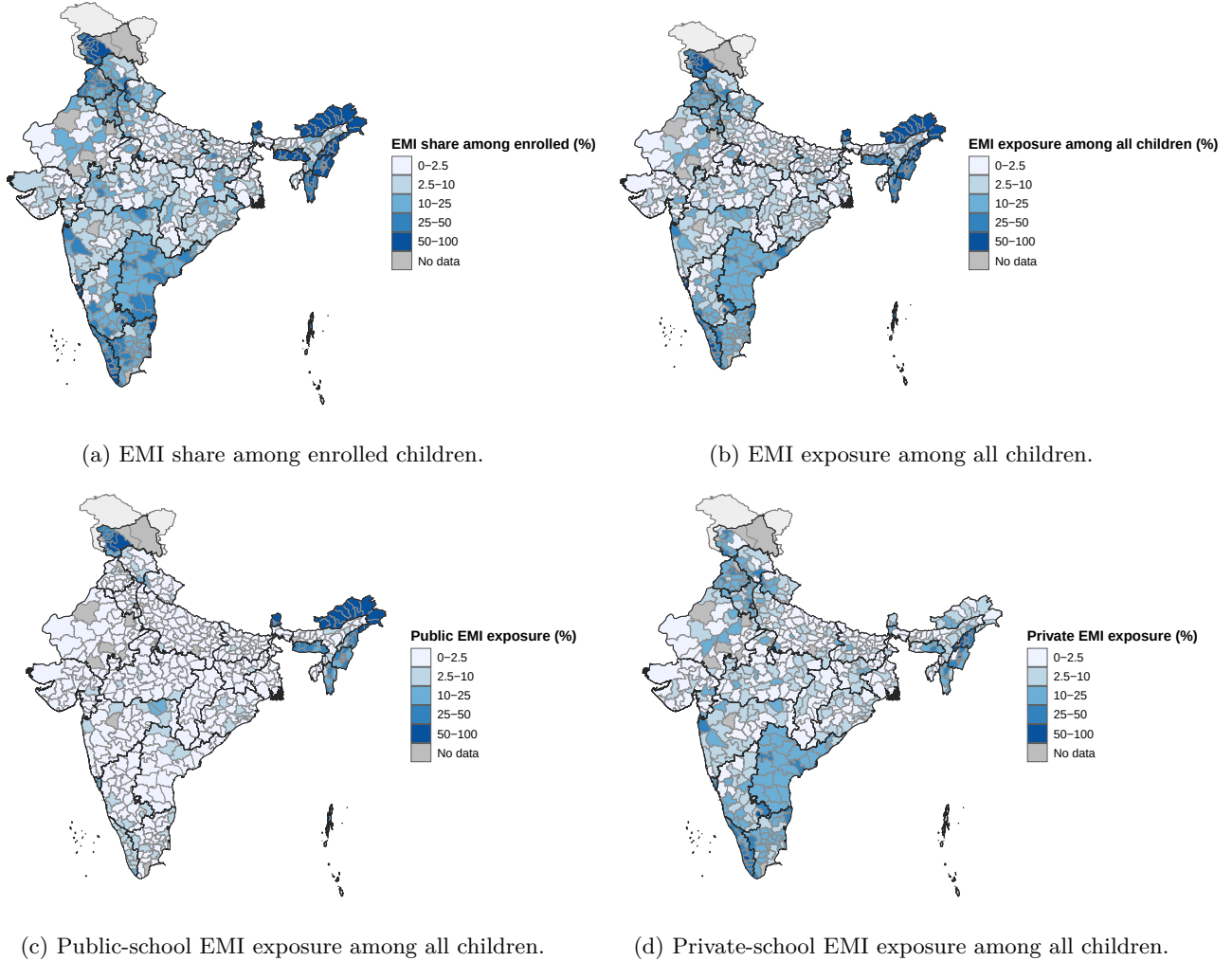
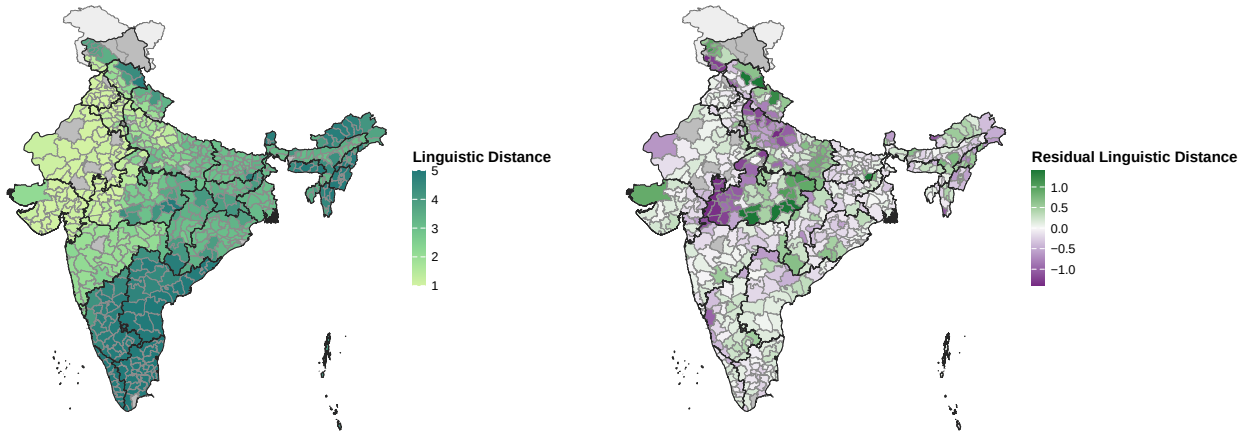


Figure 3: District distribution of English-medium schooling measures on 2001 Census boundaries. All panels use a common percentage scale. Source: NSS 64th Round, 2007–08 (National Sample Survey Office 2011). Tengshe (2014) and Natural Earth (2022) provide geospatial data. District boundaries reflect 2001 geography. Politically disputed regions are identified using Natural Earth, and those without data—Aksai Chin, Azad Kashmir, Gilgit-Baltistan, the Trans-Karakoram Tract, and Siachen Glacier—are colored light gray. Their inclusion does not imply a position on territorial disputes.



(a) Raw linguistic distance.

(b) Residual linguistic distance after state fixed effects and baseline 2001 Census covariates.

Figure 1: District linguistic distance before and after adjustment for state fixed effects and baseline 2001 Census covariates. Sources: Census of India 2001 language and district characteristics (Office of the Registrar General and Census Commissioner of India 2001; Asher et al. 2021). Tengshe (2014) and Natural Earth (2022) provide geospatial data. District boundaries reflect 2001 geography. Politically disputed regions are identified using Natural Earth, and those without data—Aksai Chin, Azad Kashmir, Gilgit-Baltistan, the Trans-Karakoram Tract, and Siachen Glacier—are colored light gray. Their inclusion does not imply a position on territorial disputes.

Assessing Weak-Instrument Strength with the Montiel Olea–Pflueger Effective F

The fitted instrumental-variable model is passed to `momentfit::MOPtest()`, using state clusters when they are available, to obtain the Montiel Olea–Pflueger effective F, critical value, p-value, and effective degrees of freedom. The table below also contains two-stage least squares (2SLS) and Anderson–Rubin results produced by the additional implementation files linked immediately before the table.

File: R/iv/weak_identification.R

```
mop_effective_f <- function(
  model, analysis_data, tau = 0.10, size = 0.05) {
  unavailable <- function(reason) {
    list(
      statistic = NA_real_, critical_value = NA_real_, p.value = NA_real_,
      effective_df = NA_real_, tau = tau, size = size,
      status = "not_estimated", reason = reason
    )
  }

  if (!inherits(model, "ivreg")) {
    return(unavailable("Montiel Olea-Pflueger effective F requires an ivreg model."))
  }

  if (!requireNamespace("momentfit", quietly = TRUE)) {
    return(unavailable("Package 'momentfit' is not installed."))
  }

  if (!is.finite(tau) || tau <= 0 || !is.finite(size) || size <= 0 || size >= 1) {
    return(unavailable("Montiel Olea-Pflueger tau and size must be valid probabilities."))
  }

  endogenous <- suppressWarnings(as.integer(model$endogenous))
  if (length(endogenous) != 1L || !is.finite(endogenous)) {
    return(unavailable("Montiel Olea-Pflueger effective F requires exactly one endogenous regressor."))
  }

  regression_formula <- tryCatch(
    stats::formula(model, component = "regressors"),
    error = function(e) NULL
  )

  instrument_formula <- tryCatch(
    stats::formula(stats::delete.response(stats::terms(model, component = "instruments"))),
    error = function(e) NULL
  )
}
```

```

)
if (is.null(regression_formula) || is.null(instrument_formula)) {
  return(unavailable("Could not recover ivreg component formulas for Montiel Olea-Pflueger effective F."))
}
needed <- unique(c(all.vars(regression_formula), all.vars(instrument_formula)))
data <- iv_analysis_frame(analysis_data, needed)
missing <- setdiff(needed, names(data))
if (length(missing)) {
  return(unavailable(paste(
    "Missing variables for Montiel Olea-Pflueger effective F:",
    paste(missing, collapse = ", ")
  )))
}
moment_data <- data[needed]
if (any(!stats::complete.cases(moment_data))) {
  return(unavailable("Montiel Olea-Pflueger analysis data contain incomplete model rows."))
}

stored_cluster <- attr(model, "cluster_state", exact = TRUE)
cluster <- if (
  length(stored_cluster) == stats::nobs(model) &&
  !anyNA(stored_cluster) && length(unique(stored_cluster)) >= 2L
) {
  as.vector(stored_cluster)
} else {
  iv_model_cluster(model, data)
}
covariance <- if (is.null(cluster)) "MDS" else "CL"
covariance_options <- if (is.null(cluster)) {
  list(type = "HCO")
} else {
  # The momentfit::MOptest() function uses an HCO clustered moment covariance with its own
  # finite-cluster adjustment. The conventional first-stage Wald F is computed
  # separately with the HC1 covariance used for the first-stage regression.
  list(
    cluster = data.frame(cluster = cluster),
    type = "HCO", cadjust = TRUE, multi0 = FALSE
  )
}

result <- tryCatch({

```

```

moment_model <- momentfit::momentModel(
  regression_formula,
  instrument_formula,
  data = moment_data,
  vcov = covariance,
  vcovOptions = covariance_options
)
momentfit::MOPtest(
  moment_model,
  tau = tau,
  size = size,
  estMethod = "TSLS",
  simplified = TRUE,
  print = FALSE
)
}, error = function(e) e)
if (inherits(result, "error")) return(unavailable(conditionMessage(result)))

values <- suppressWarnings(as.numeric(result[c("Feff", "critValue", "pvalue", "Keff"))]))
names(values) <- c("statistic", "critical_value", "p.value", "effective_df")
if (length(values) != 4L || any(!is.finite(values))) {
  return(unavailable("momentfit::MOPtest() did not return finite effective-F statistics."))
}

c(
  as.list(values),
  list(tau = tau, size = size, status = "estimated", reason = NA_character_)
)
}

```

Additional code used for this table: R/iv/estimate_2sls.R, R/iv/consumption_outcome_registry.R, R/output/appendix_identification_exhibits.R, R/output/save_tables.R.

Table 10: Inference with Weak Instruments for Long-Run Consumption Change

	Real log consumption change, 2004-05 to 2022-23	Real log consumption change, 2004-05 to 2023-24
EMI exposure	0.117 (0.182)	0.137 (0.281)
MOP effective F	0.69	0.41
AR p -value at zero	0.029	0.081
AR 95% accepted set	$[-0.193, -0.030]^{\dagger} \cup [0.009, 0.193]^{\dagger}$	$[-0.186, 0.186]^{\dagger}$
Direct effect threshold for $\beta = 0$ (share of reduced form)	16.6%	0.0%
Observations	524	522

The table reports conventional 2SLS coefficients with state-clustered standard errors. Instrument strength is summarized by the Montiel Olea–Pflueger effective F statistic, and Anderson–Rubin tests provide inference robust to weak instruments. The 95% AR accepted sets are displayed as unions of closed intervals. A dagger marks a component that reaches an evaluated search boundary. Its outer endpoint is the numerical search limit and should not be interpreted as a finite confidence bound. The direct effect threshold is the smallest permitted direct effect, expressed as a share of the absolute reduced form, for which the AR set includes $\beta = 0$. Sources: Census of India 2001; NSS 61st Round, 2004–05; NSS 64th Round, 2007–08; HCES 2022–23; HCES 2023–24.

Linking Rural and Urban State CPI Series Across Index Bases

Old and new rural and urban state CPI series are linked over their January 2013–December 2014 overlap using the median monthly ratio of the new index to the old index. At least six common months are required, and the returned data retain the overlap counts and ratio dispersion used to assess sensitivity to the link period.

File: R/prices/build_temporal_price_series.R

```
build_temporal_price_series <- function(
  price_sources,
  switch_date = as.Date("2013-01-01"),
  overlap_start = as.Date("2013-01-01"),
  overlap_end = as.Date("2014-12-01"),
  minimum_link_months = 6L,
  pre_switch_start = NULL,
  pre_switch_end = NULL) {
  switch_date <- price_boundary(switch_date)
  old <- select_pre_2013_price_series(price_sources)
  new <- select_post_2013_price_series(price_sources)
  links <- summarise_price_links(old, new, overlap_start, overlap_end)

  minimum_link_months <- as.integer(minimum_link_months)
  if (length(minimum_link_months) != 1L || is.na(minimum_link_months) || minimum_link_months < 1L) {
```

```

    stop("minimum_link_months must be one positive integer.", call. = FALSE)
}

old_groups <- unique(old[old$period < switch_date, c("state_code", "sector"), drop = FALSE])
linked_groups <- links[links$link_months >= minimum_link_months, c("state_code", "sector"), drop = FALSE]
linked_key <- paste(linked_groups$state_code, linked_groups$sector, sep = "\r")
missing_key <- !paste(old_groups$state_code, old_groups$sector, sep = "\r") %in% linked_key
if (any(missing_key)) {
  bad <- old_groups[missing_key, , drop = FALSE]
  stop(
    "Pre-2013 price series lack a sufficient direct link to CPI-R/U: ",
    paste(paste(bad$state_code, bad$sector, sep = "/"), collapse = ", "),
    call. = FALSE
  )
}

old <- merge(old, links, by = c("state_code", "sector"), all.x = TRUE, sort = FALSE)
old <- old[old$period < switch_date, , drop = FALSE]
if (!is.null(pre_switch_start) || !is.null(pre_switch_end)) {
  if (is.null(pre_switch_start) || is.null(pre_switch_end)) {
    stop("Both pre_switch_start and pre_switch_end are required when trimming the old price series.",
      call. = FALSE)
  }
  pre_switch_start <- price_boundary(pre_switch_start)
  pre_switch_end <- price_boundary(pre_switch_end)
  if (pre_switch_end < pre_switch_start || pre_switch_end >= switch_date) {
    stop("The pre-switch production window must end before the switch date.", call. = FALSE)
  }
  old <- old[old$period >= pre_switch_start & old$period <= pre_switch_end, , drop = FALSE]
}

old$index_unlinked <- old$index
old$index <- num(old$index) * num(old$link_factor)

new <- new[new$period >= switch_date, , drop = FALSE]
new$link_factor <- 1
new$link_months <- NA_integer_
new$link_ratio_mad <- NA_real_
new$link_ratio_min <- NA_real_
new$link_ratio_max <- NA_real_
new$index_unlinked <- new$index

```

```

columns <- c(
  "state_code", "sector", "period", "index", "index_unlinked", "price_source",
  "link_factor", "link_months", "link_ratio_mad", "link_ratio_min", "link_ratio_max"
)
index <- rbind(old[columns], new[columns])
index$year <- as.integer(format(index$period, "%Y"))
index$month <- as.integer(format(index$period, "%m"))
index <- index[order(index$state_code, index$sector, index$period), , drop = FALSE]
rownames(index) <- NULL
validate_temporal_price_chain(index, switch_date)

structure(
  list(index = index, links = links, switch_date = switch_date),
  class = "emi_temporal_price_series"
)
}

```

Rural and Urban State Price Deflators for Comparable Real Consumption

The linked rural and urban CPI series is combined with state-specific price relatives for rural and urban areas derived from Tendulkar poverty lines. The resulting deflators express survey consumption in comparable 2011–12 rupees; the figure below compares district estimates across the 2022–23 and 2023–24 Household Consumption Expenditure Survey (HCES) rounds.

File: R/prices/price_deflators.R

```

build_state_sector_deflator <- function(
  temporal_index, spatial_relatives, reference_period = NULL, reference_index = NULL) {
  idx <- safe_df(temporal_index)
  spatial <- safe_df(spatial_relatives)
  validate_price_index(idx, keys = c("state_code", "sector", "period"))
  required_spatial <- c("state_code", "sector", "spatial_price_relative")
  missing <- setdiff(required_spatial, names(spatial))
  if (length(missing)) {
    stop("Spatial price relatives are missing columns: ", paste(missing, collapse = ", "), call. = FALSE)
  }
  if (anyDuplicated(spatial[c("state_code", "sector")])) {
    stop("Spatial price relatives must have one row per state and sector.", call. = FALSE)
  }
  if (any(!positive_finite(num(spatial$spatial_price_relative)))) {

```

```

    stop("Spatial price relatives must be positive and finite.", call. = FALSE)
  }

  if (is.null(reference_index)) {
    if (is.null(reference_period)) stop("A price reference period or reference-index table is required.",
    ↪ call. = FALSE)
    reference_period <- as.Date(reference_period)
    ref <- idx[idx$period %in% reference_period, , drop = FALSE]
    if (!nrow(ref)) stop("No observations fall in the requested price reference period.", call. = FALSE)
    ref_mean <- stats::aggregate(index ~ state_code + sector, ref, function(x) mean(num(x), na.rm = TRUE))
    names(ref_mean)[names(ref_mean) == "index"] <- "reference_index"
  } else {
    ref_mean <- safe_df(reference_index)
    required_reference <- c("state_code", "sector", "reference_index")
    missing_reference <- setdiff(required_reference, names(ref_mean))
    if (length(missing_reference)) {
      stop("Reference-price table is missing columns: ", paste(missing_reference, collapse = ", "), call. =
      ↪ FALSE)
    }
    if (anyDuplicated(ref_mean[c("state_code", "sector")]) ||
    ↪ any(!positive_finite(ref_mean$reference_index))) {
      stop("Reference-price table must have one positive row per state and rural/urban sector.", call. =
      ↪ FALSE)
    }
    ref_mean <- ref_mean[required_reference]
  }

  out <- merge(
    idx,
    spatial,
    by = c("state_code", "sector"), all.x = TRUE, sort = FALSE
  )

  out <- merge(out, ref_mean, by = c("state_code", "sector"), all.x = TRUE, sort = FALSE)
  if ("source_state_code" %in% names(out) && any(!positive_finite(out$reference_index))) {
    donor_reference <- ref_mean
    names(donor_reference)[names(donor_reference) == "state_code"] <- "source_state_code"
    names(donor_reference)[names(donor_reference) == "reference_index"] <- "donor_reference_index"
    out <- merge(out, donor_reference, by = c("source_state_code", "sector"), all.x = TRUE, sort = FALSE)
    missing_reference <- !positive_finite(out$reference_index)
    out$reference_index[missing_reference] <- out$donor_reference_index[missing_reference]
    out$donor_reference_index <- NULL
  }
}

```

```

out$temporal_price_relative <- num(out$index) / num(out$reference_index)
out$price_deflator <- num(out$spatial_price_relative) * num(out$temporal_price_relative)
if (any(!positive_finite(out$price_deflator))) {
  stop("Deflator construction left missing or invalid values.", call. = FALSE)
}
out <- out[order(out$state_code, out$sector, out$period), , drop = FALSE]
rownames(out) <- NULL
out
}

build_state_sector_price_deflators <- function(
  temporal_series,
  state_rules = read_price_state_crosswalk(),
  poverty_lines = read_tendulkar_poverty_lines(),
  reference_period = seq(as.Date("2011-07-01"), as.Date("2012-06-01"), by = "month"),
  reference_index = NULL,
  start_period = NULL,
  end_period = NULL,
  reference_rupees = tendulkar_real_poverty_line(),
  require_complete_grid = TRUE) {
  temporal_index <- if (inherits(temporal_series, "emi_temporal_price_series")) {
    temporal_series$index
  } else {
    safe_df(temporal_series)
  }
  expanded <- apply_price_state_rules(
    temporal_index,
    state_rules,
    start_period = start_period,
    end_period = end_period,
    require_complete_grid = require_complete_grid
  )
  spatial <- build_tendulkar_spatial_relatives(poverty_lines, reference_rupees)
  missing_spatial <- setdiff(
    unique(paste(expanded$state_code, expanded$sector, sep = "\r")),
    unique(paste(spatial$state_code, spatial$sector, sep = "\r"))
  )
  if (length(missing_spatial)) {
    stop("Tendulkar poverty-line data do not cover every state and rural/urban CPI series.", call. = FALSE)
  }
  build_state_sector_deflator(

```

```

    expanded,
    spatial,
    reference_period = reference_period,
    reference_index = reference_index
  )
}

```

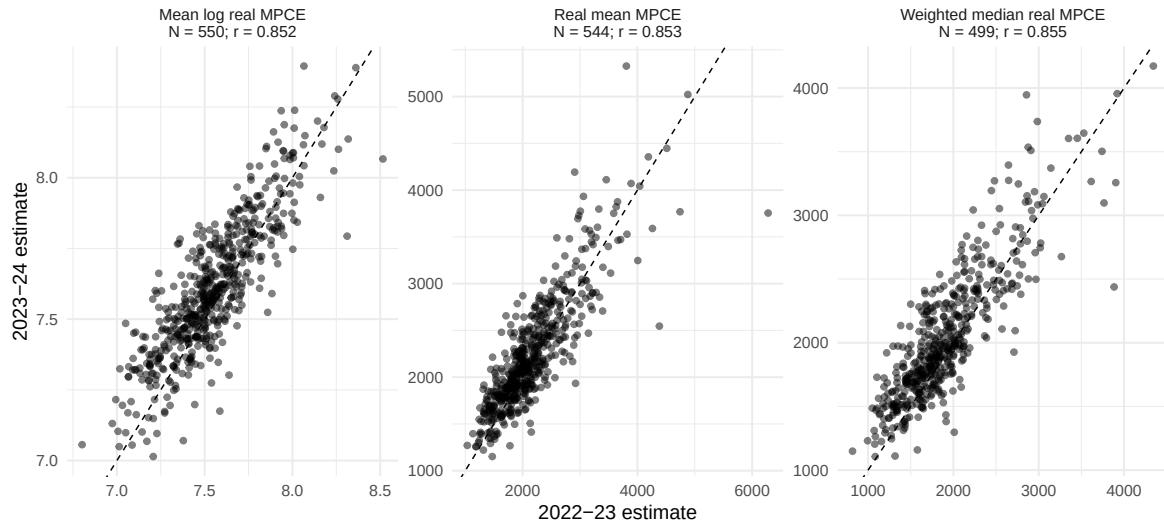


Figure 8: Agreement between district real-consumption estimates in the 2022–23 and 2023–24 HCES rounds. Each panel uses districts retained in both rounds under the consumption sample criteria; the dashed line is the 45-degree line. Sources: National Sample Survey Office, HCES 2022–23 and HCES 2023–24 (National Sample Survey Office 2024, 2025).

Reusing One Analysis Definition from Raw and Processed Inputs

The analysis that begins from raw inputs and the replication that begins from processed data call the same `{targets}` definitions. These targets assemble the consumption panel and estimate the same instrumental-variable specifications; related targets verify outcome availability and construct the schooling-consumption comparisons.

File: `R/pipeline/core_consumption_iv_targets.R`

```

consumption_iv_analysis_target_definitions <- function() {
  list(
    tar_target(
      consumption_iv_panel,

```

```

    attach_consumption_iv_outcomes(
      district_panel,
      consumption_district_welfare,
      consumption_iv_outcome_registry
    )
  ),
  tar_target(
    consumption_iv_outcome_coverage,
    validate_consumption_iv_outcome_coverage(
      summarize_consumption_iv_outcome_coverage(
        consumption_iv_panel,
        consumption_iv_specifications
      )
    )
  ),
  tar_target(
    consumption_iv_dynamics,
    {
      consumption_iv_outcome_coverage
      validate_consumption_iv_dynamics(
        estimate_consumption_iv_dynamics(
          consumption_iv_panel,
          consumption_iv_specifications,
          cfg
        ),
        consumption_iv_specifications
      )
    }
  ),
  tar_target(
    schooling_consumption_bridge,
    diagnose_schooling_consumption_bridge(
      consumption_iv_panel,
      consumption_iv_outcome_registry,
      census_2001_control_registry
    )
  ),
  tar_target(
    schooling_consumption_conversion,
    diagnose_schooling_consumption_conversion(
      consumption_iv_panel,

```

```
        consumption_iv_outcome_registry,  
        census_2001_control_registry  
    )  
)  
}
```